

Test Unitaire en React / Redux en utilisant Jest

1. Définition

Un **test unitaire** vérifie qu'une petite partie de ton code (une fonction, un composant) fonctionne **correctement toute seule**.

Exemple :

- Vérifier qu'un bouton s'affiche
- Vérifier qu'un clic appelle une fonction
- Vérifier qu'une action Redux retourne le bon objet

Pourquoi tester ?

- Éviter les bugs
- Être sûr que ton code fonctionne après une modification
- Avoir confiance quand tu ajoutes des fonctionnalités

2. Les outils que nous allons utiliser

Jest

- Framework de test JavaScript
- Permet d'écrire des tests et de faire des assertions

React Testing Library (RTL)

- Teste les composants **comme un utilisateur**
- Travaille avec le DOM (texte, boutons, formulaires)

3. Installation (projet React)

Si tu utilises **Create React App**, Jest est déjà installé

Sinon :

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom
```

4. Structure des fichiers de test

Un fichier de test se termine par :

- .test.js ou .spec.js

Exemple :

```
Button.js  
Button.test.js
```

5. Tester un composant React simple

Composant à tester

```
// Button.js  
function Button({ label }) {  
  return <button>{label}</button>;  
}  
  
export default Button;
```

Test du composant

```
// Button.test.js  
import { render, screen } from '@testing-library/react';  
import Button from './Button';  
  
test('affiche le texte du bouton', () => {  
  render(<Button label="Clique ici" />);  
  
  const button = screen.getByText('Clique ici');  
  expect(button).toBeInTheDocument();  
});
```

Explication

- `render()` → affiche le composant
- `screen.getByText()` → cherche du texte à l'écran
- `expect()` → vérifie le résultat

6. Tester une interaction (clic)

Composant

```
function Counter({ onIncrement }) {  
  return <button onClick={onIncrement}>+</button>;  
}
```

Test

```
import { render, screen, fireEvent } from '@testing-library/react';

test('appelle la fonction au clic', () => {
  const mockClick = jest.fn();

  render(<Counter onIncrement={mockClick} />);

  fireEvent.click(screen.getByText('+'));

  expect(mockClick).toHaveBeenCalled();
});
```

À retenir

- `jest.fn()` crée une **fausse fonction** (mock)
- `fireEvent.click()` simule une action utilisateur
- Introduction à Redux (rappel rapide)

7. Une action Redux

```
// actions.js
export const increment = () => ({
  type: 'INCREMENT'
});
```

8. Tester une action Redux

Test de l'action

```
import { increment } from './actions';

test('increment retourne la bonne action', () => {
  const action = increment();

  expect(action).toEqual({
    type: 'INCREMENT'
  });
});
```

Ici on vérifie le retour de la fonction, **rien de plus.**

9. Tester un reducer Redux

Reducer

```
// reducer.js
const initialState = { count: 0 };

function reducer(state = initialState, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    default:
      return state;
  }
}

export default reducer;
```

Test

```
import reducer from './reducer';

test('incrémente le compteur', () => {
  const state = reducer({ count: 0 }, { type: 'INCREMENT' });

  expect(state.count).toBe(1);
});
```

10. Exemple d'application 1 : CRUD App React + Redux Toolkit + Tests

a. Fonctionnalités de l'application

Nous allons gérer une **liste de produits** avec :

- Ajouter un produit
- Modifier un produit
- Supprimer un produit
- Afficher la liste

Chaque produit aura :

```
{ id, name, price }
```

b. Installation des dépendances

```
npm install @reduxjs/toolkit react-redux react-router-dom bootstrap
npm install --save-dev @testing-library/react @testing-library/jest-dom
```

c. Dans index.js :

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

d. Structure du projet

```
src/  
├── app/  
│   └── store.js  
├── features/products/  
│   ├── productSlice.js  
│   └── productSlice.test.js  
├── components/  
│   ├── ProductList.js  
│   ├── ProductForm.js  
│   └── ProductList.test.js  
├── App.js  
└── index.js
```

e. Store Redux

```
// app/store.js  
import { configureStore } from '@reduxjs/toolkit';  
import productReducer from '../features/products/productSlice';  
  
export const store = configureStore({  
  reducer: {  
    products: productReducer  
  }  
});
```

f. Slice Redux Toolkit (CRUD)

```
// features/products/productSlice.js  
import { createSlice, nanoid } from '@reduxjs/toolkit';  
  
const initialState = [];  
  
const productSlice = createSlice({  
  name: 'products',  
  initialState,  
  reducers: {  
    addProduct: {  
      reducer(state, action) {  
        state.push(action.payload);  
      },  
      prepare(product) {  
        return {  
          payload: product,  
          meta: {  
            id: nanoid(),  
            createdAt: new Date().toISOString(),  
            updatedAt: new Date().toISOString(),  
          },  
        };  
      },  
    },  
  },  
});
```

```

payload: { ...product, id: nanoid() }
};
}
},
removeProduct(state, action) {
return state.filter(p => p.id !== action.payload);
},
editProduct(state, action) {
const index = state.findIndex(p => p.id === action.payload.id);
state[index] = action.payload;
}
}
});

export const { addProduct, removeProduct, editProduct } = productSlice.actions;
export default productSlice.reducer;

```

g. Composant ProductList

```

// components/ProductList.js
import { useSelector, useDispatch } from 'react-redux';
import { removeProduct } from '../features/products/productSlice';

function ProductList() {
const products = useSelector(state => state.products);
const dispatch = useDispatch();

return (
<div className="container mt-3">
<h2>Produits</h2>
<ul className="list-group">
{products.map(p => (
<li key={p.id} className="list-group-item d-flex justify-content-between">
{p.name} - {p.price} €
<button
className="btn btn-danger btn-sm"
onClick={() => dispatch(removeProduct(p.id))}
>
Supprimer
</button>
</li>
))}
</ul>
</div>
);
}

export default ProductList;

```

h. Composant ProductForm

```
// components/ProductForm.js
import { useDispatch } from 'react-redux';
import { addProduct } from '../features/products/productSlice';
import { useState } from 'react';

function ProductForm() {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const dispatch = useDispatch();

  const handleSubmit = e => {
    e.preventDefault();
    dispatch(addProduct({ name, price }));
    setName("");
    setPrice("");
  };

  return (
    <form onSubmit={handleSubmit} className="container mt-3">
      <input
        className="form-control mb-2"
        placeholder="Nom"
        value={name}
        onChange={e => setName(e.target.value)}
      />
      <input
        className="form-control mb-2"
        placeholder="Prix"
        value={price}
        onChange={e => setPrice(e.target.value)}
      />
      <button className="btn btn-primary">Ajouter</button>
    </form>
  );
}

export default ProductForm;
```

i. Routage

```
// App.js
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import ProductList from '../components/ProductList';
import ProductForm from '../components/ProductForm';

function App() {
```

```

return (
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<ProductList />} />
      <Route path="/add" element={<ProductForm />} />
    </Routes>
  </BrowserRouter>
);
}

export default App;

```

j. Tests du reducer (Redux Toolkit)

```

// productSlice.test.js
import reducer, { addProduct, removeProduct } from './productSlice';

test('ajoute un produit', () => {
  const state = reducer([], addProduct({ name: 'Livre', price: 10 }));
  expect(state.length).toBe(1);
  expect(state[0].name).toBe('Livre');
});

test('supprime un produit', () => {
  const initialState = [{ id: '1', name: 'Stylo', price: 2 }];
  const state = reducer(initialState, removeProduct('1'));
  expect(state.length).toBe(0);
});

```

k. Test d'un composant (ProductList)

```

// ProductList.test.js
import { render, screen } from '@testing-library/react';
import { Provider } from 'react-redux';
import { configureStore } from '@reduxjs/toolkit';
import productReducer from '../features/products/productSlice';
import ProductList from './ProductList';

test('affiche un produit', () => {
  const store = configureStore({
    reducer: { products: productReducer },
    preloadedState: {
      products: [{ id: '1', name: 'PC', price: 1000 }]
    }
  });

  render(
    <Provider store={store}>
      <ProductList />
    </Provider>
  );
});

```



```
render(  
  <Provider store={store}>  
    <ProductList />  
  </Provider>  
)  
  
expect(screen.getByText(/PC/)).toBeInTheDocument()  
});
```